CURSOR HACKATHON · 8-HOUR BUILD · HOW IT WORKS

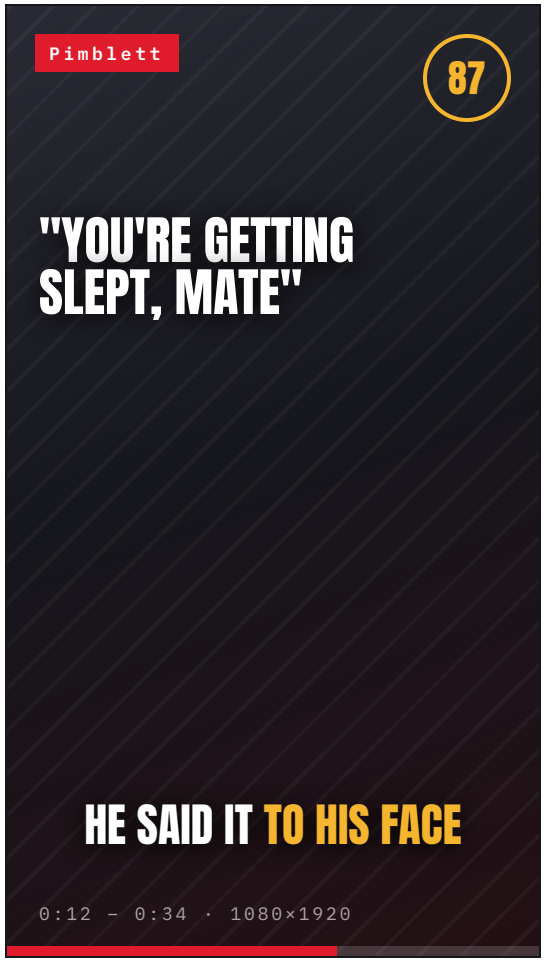
HOOKSHOT

Drop a video. Get back ranked, captioned, ready-to-post vertical clips.

THE VIDEO NEVER LEAVES YOUR BROWSER

NEXT.JS 16 | FFMPEG.WASM | WHISPER-1 | GPT-5 | CANVAS + MEDIARECORDER | 0 BUCKETS

An OpusClip-style clipping machine that finds the postable moments in long footage, writes the copy, burns the captions and renders the vertical cut — without a single frame of your video ever touching a server.



CLIPPING IS A GRIND. AND EVERY TOOL MAKES YOU UPLOAD FIRST.

A 90-minute press conference holds maybe six postable moments. Finding them, cutting them, reframing them and writing the copy is a whole afternoon — and on every existing tool, that afternoon starts with a progress bar.

01
SCRUBBING BY HAND

Someone drags a playhead across ninety minutes of talking, hunting for the six seconds where the room went quiet. There is no index for "that was the line."

02
THE UPLOAD TAX

Every clip service wants your 2 GB file on their servers before it does anything. You wait on your uplink, then their queue — and your unreleased footage now lives on someone else's disk.

03
THEN THE BUSYWORK

Cut. Reframe to 9:16. Caption every word. Write a hook, a caption and hashtags per platform. Repeat six times, per video, per week.

BY THE TIME THE CLIP IS POSTED, THE MOMENT HAS ALREADY MOVED ON.

The window on fight-week content is hours, not days. Whatever the workflow costs in latency,
it costs in reach.

THE VIDEO NEVER LEAVES THE BROWSER.

BROWSER — EVERYTHING THAT TOUCHES PIXELS

Source mp4	ffmpeg.wasm extract	Merge + snap	Cut & caption	Export MP4 · WebM

↓ AUDIO CHUNKS < 4 MB · TEXT ↑

SERVER — AUDIO AND TEXT ONLY, THREE THIN ROUTES

/api/youtube	/api/transcribe	/api/moments
`yt-dlp stream`	`whisper-1`	`gpt-5 JSON`

Never crosses the wire: the source video. No object storage, no upload queue, no transcode farm, no egress bill. Nothing to leak.

- **Work starts on drop.** No upload wait — audio extraction begins the instant the file lands, on a file that never moves.
- **It fits the platform by design.** Vercel caps request bodies at 4.5 MB. Audio chunks are engineered to 1.4 MB. The limit stopped being a problem and became the spec.
- **Zero infrastructure.** Three stateless route handlers. No database, no bucket, no worker queue, no job table — so there is nothing to operate and nothing to bill.
- **Your footage stays yours.** Unreleased promo material never lands on a third-party disk, because it never leaves the tab.

EVERYONE ELSE MOVES THE VIDEO TO THE COMPUTE. WE MOVED THE COMPUTE TO THE VIDEO.

THE PIPELINE

Drop to downloadable clip. The only things that ever cross the network are compressed audio going out, and text coming back.

STAGE 01
INGEST
Drop an mp4, or paste a YouTube link and let yt-dlp stream it straight through the route — the server keeps nothing.

BROWSER

local file

STAGE 02
EXTRACT
ffmpeg.wasm loads in the tab and rips mono 32 kbps audio, segmented into ~6-minute chunks under 4 MB.

BROWSER · WASM

stays local

STAGE 03
TRANSCRIBE
Chunks post to whisper-1, three in flight, returning word-level timestamps and segments.

SERVER · WHISPER

↑↓
audio out · words back

STAGE 04
MERGE
Chunk offsets prefix-summed from real durations, forced monotonic, snapped to sentence edges. Pure code.

BROWSER · CODE

stays local

STAGE 05
JUDGE
The text timeline goes to gpt-5, which picks the moments worth posting and writes the copy. Output is validated field by field.

SERVER · GPT-5

↑↓
text out · JSON back

STAGE 06
CUT & EXPORT
Keyframe-aligned `-c copy` MP4 with a matched .srt, or a 1080×1920 canvas render with captions burned in.

BROWSER · CANVAS

stays local

A 2 GB SOURCE BECOMES ABOUT 10 MB OF AUDIO AND A FEW KILOBYTES OF JSON. THAT IS THE ENTIRE NETWORK COST.

ENGINEERED AGAINST A 4.5 MB WALL

Vercel rejects request bodies over 4.5 MB. Rather than route around the limit, the audio format was derived from it — every constant below is reverse-engineered from that one number.

```
# one pass, in the browser, no server involved
ffmpeg -i in.mp4 -vn -ac 1 -ar 16000 -b:a 32k \
 -f segment -segment_time 360 chunk%03d.mp3
```

360s	1.4MB	4MB	3
SEGMENT	TYPICAL CHUNK	HARD CEILING	IN FLIGHT

DO THE ARITHMETIC

A 40-minute presser becomes seven files of roughly 1.4 MB — about 10 MB total, uploaded in parallel while the tab stays responsive. The 2 GB source never moves an inch. Every chunk is size-checked before it is sent; if mp3 encoding is unavailable, the fallback drops to 2-minute 16 kHz WAV, which lands at 3.8 MB — still inside the wall.

- **Lazy, self-hosted, single-thread.** The 32 MB wasm core loads only after a file drops, behind a progress bar. Single-thread on purpose — the multithreaded core would demand COOP/COEP headers.
- **Offsets come from reality, not arithmetic.** Chunk start times are prefix-summed from the durations Whisper actually reports, not the nominal 360-second grid — encoder drift would otherwise slide captions late.
- **Timestamps are forced monotonic.** Whisper occasionally emits a word that starts before the previous one ended. Code repairs the sequence before anything downstream reads it.
- **The failure modes are handled.** No audio track, a stale core, a chunk that busts the ceiling — each has an explicit branch and a readable error, because a dead dropzone in a demo is fatal.

THE PLATFORM LIMIT DIDN'T GET WORKED AROUND. IT BECAME THE FORMAT.

THE MODEL JUDGES. THE CODE COMPUTES.

The split is absolute, and it is the reason this holds together. A language model is excellent at deciding which twenty seconds are worth posting, and unreliable at arithmetic. So it never does any.

THE MODEL DECIDES

- Which twenty seconds are worth posting
- The hook line that opens the clip
- Caption, hashtags, and the audience read
- Honest compliance flags on every clip

THE CODE DECIDES

- Chunk offsets and timestamp merging
- Snapping cuts to real sentence boundaries
- Keyframe-aligned cutting and concatenation
- Subtitle timing across every jump cut

NOTHING THE MODEL RETURNS IS TRUSTED

Every field is coerced server-side. Scores clamped to range. Spans clamped in-bounds, sorted, overlaps merged, anything under 4 seconds dropped, capped at three spans and 120 seconds. Missing compliance data defaults to the cautious value, never the convenient one.

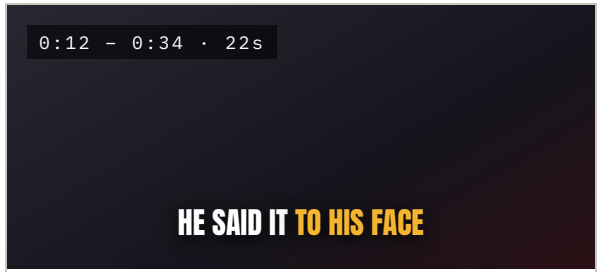
THE HASHTAG IS APPENDED IN CODE

Campaign rules require every caption to end with `#UFCClips`. The prompt asks for it — then `ensureUFCClipsHashtag()` appends it anyway if it's missing. A prompt is a request; code is a guarantee.

EVERY NUMBER IN THIS PRODUCT IS COMPUTED.
NOT ONE OF THEM IS GUESSED BY A LANGUAGE MODEL.

WHAT COMES BACK

Five to eight clips, ranked — so the best one is on top, not the first one found.



"YOU'RE GETTING SLEPT, MATE"

HOOK	92
EMOTION	84
QUOTABLE	90
LOOP	78

Paddy never blinked. Ilia had no answer. **#UFCClips** #UFC329 #Pimblett

⚠ WALKOUT RISK — BROADCAST FOOTAGE

1 **RANKED BY A FOUR-PART SCORE**

Hook, emotion, quotability, loopability. The total orders the grid, so a reviewer reads top-down instead of hunting.

2 **CUTS LAND ON SENTENCES**

The model proposes rough bounds; code snaps them to real sentence edges using word timestamps. Clips never open mid-word or die on a half-finished thought.

3 **JUMP CUTS REMOVE THE DEAD AIR**

When a setup and its payoff sit forty seconds apart, the clip is assembled from up to three non-adjacent spans, stitched with a fast white flash. Tight arc, no filler.

4 **COPY THAT KNOWS WHO IT'S FOR**

Name your audience or leave it blank and the model infers who would share this from the transcript itself — then tunes hook, caption and hashtags to them.

5 **COMPLIANCE STAYS VISIBLE**

In-fight broadcast risk, walkout risk and low-value risk ride on every clip and render as a banner. The tool flags its own risky output rather than hiding it.

TWO WAYS OUT. BOTH RENDERED IN THE TAB.

MP4 + .SRT — WORKS EVERYWHERE

LOSSLESS, INSTANT, SOURCE RESOLUTION

A keyframe-aligned `-c copy` cut: no re-encode, so it finishes in about a second and loses nothing. The sidecar .srt is timed against the cut's real timeline. Multi-span clips are cut per segment and stream-copy concatenated.

WEBM 1080×1920 — THE POST-READY SHORT

VERTICAL, CAPTIONS BURNED IN

A canvas compositor draws the frame contained over a blurred cover fill — nothing cropped — with karaoke captions, the hook title over the first three seconds, and a progress bar, captured through MediaRecorder. No server render.

THE HARD PART — AND IT IS GENUINELY HARD

STREAM COPY CAN ONLY BEGIN ON A KEYFRAME

Ask ffmpeg to cut at 12.4s and the MP4 really starts at the keyframe at or before it — maybe 11.1s. Naive captioning fires early and drifts further with every jump cut. So the keyframe is probed with ffprobe, the true start is returned with the blob, and subtitles are timed against a virtual timeline built from the real segment lengths:

```
offset[i] = sum over j < i of ( end[j] − actualStart[j] )
a word at source time t in segment i lands at ( t − actualStart[i] ) + offset[i]  // cues never straddle a segment edge
```

Captions line up exactly — across every jump cut, on every player. And exports carry no logo and no watermark. Ever.

WHAT IT COST, AND WHAT IT STILL CAN'T DO

DECISIONS THAT PAID

- **76s → 23s on moment analysis.** Default reasoning effort was burning 75 seconds on a short transcript and creeping toward the function timeout. Dropping to minimal effort made it both fast and safe — latency and reliability turned out to be the same fix.
- **A model chain, not a model.** gpt-5 with a gpt-5-mini fallback: if the primary errors or returns nothing usable, the next model runs before the user ever sees a failure.
- **Cuts are serialized.** ffmpeg.wasm is a singleton with one filesystem and fixed output names, so two clip cards exporting at once would silently clobber each other. A promise queue makes that impossible.
- **Every route is rate-limited.** A per-IP sliding window in front of all three handlers, because a public demo and an OpenAI key are a bad pair without one.

WHAT IT HONESTLY CAN'T DO YET

- **WebM export is Chrome-first.** It rides on MediaRecorder, which is flaky in Safari. The MP4 + .srt path is the fallback that always works.
- **MP4 cuts can open early.** Keyframe spacing is the source's business, so a clip may start up to a couple of seconds before the requested point. Captions stay correct; the video just has a short runway.
- **The rate limiter is per-instance.** In-memory, so each warm serverless instance keeps its own window. It guards a budget; it is not a distributed defense.
- **Memory is the real ceiling.** ffmpeg.wasm holds the whole file in the tab, so link pulls are capped at 500 MB and very long sources need trimming first.

NOTHING HERE IS A STUB.
EVERY PATH IN THE DEMO IS THE REAL PATH.

~3.8k	3	0	0 bytes
LINES OF TYPESCRIPT	API ROUTES, ALL THIN	DATABASES · BUCKETS · QUEUES	OF VIDEO STORED, EVER

DROP A TAPE. WATCH IT WORK.

01
DROP OR PASTE
An mp4 goes in, or a YouTube link. ffmpeg.wasm wakes up behind a progress bar and starts ripping audio immediately.

02
WATCH THE STAGES
Extract, transcribe chunk by chunk, merge, judge. Every stage narrates itself, so the wait never feels like a hang.

03
READ THE GRID
Ranked clip cards, each with a live karaoke preview, a hook title, a ready-to-paste caption and its compliance flags.

04
EXPORT AND POST
One click for a vertical short with captions burned in, or MP4 + .srt for an editor. Both render in the tab.

AND THE VIDEO NEVER LEFT THE BROWSER.

HOOKSHOT · BUILT IN 8 HOURS · NEXT.JS 16 · FFMPEG.WASM · WHISPER-1 · GPT-5